

✓ 1 Even-Odd Printing Using Two Threads

Goal:

- Thread A → prints even numbers
- Thread B → prints odd numbers
- Output: **1 2 3 4 5 6 ...**

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>

int counter = 1;
int n = 10;
std::mutex m;
std::condition_variable cv;

void printOdd() {
    while (counter <= n) {
        std::unique_lock<std::mutex> lock(m);
        cv.wait(lock, [] { return counter % 2 == 1; });

        if (counter <= n)
            std::cout << "Odd : " << counter << std::endl;

        counter++;
        cv.notify_all();
    }
}

void printEven() {
    while (counter <= n) {
        std::unique_lock<std::mutex> lock(m);
        cv.wait(lock, [] { return counter % 2 == 0; });

        if (counter <= n)
            std::cout << "Even : " << counter << std::endl;

        counter++;
        cv.notify_all();
    }
}

int main() {
    std::thread t1(printOdd);
    std::thread t2(printEven);

    t1.join();
    t2.join();
}
```

Thread-1: Print Odd Numbers

```
void printOdd() {
    while (counter <= n) {
        std::unique_lock<std::mutex> lock(m);
        cv.wait(lock, [] { return counter % 2 == 1; });
```

Runs until all numbers printed.

✓ Lock the mutex so only one thread accesses counter.

- Thread waits until **counter is odd**.
- If not odd, the thread sleeps (does not spin).

Wake Up Other Thread

```
        cv.notify_all();
```

✓ Signals even thread:
"Hey, I finished printing odd; now you check your turn."

Thread-2: Print Even Numbers

Works exactly like odd thread, but waits for **counter to be even**.

```
void printEven() {
    while (counter <= n) {
        std::unique_lock<std::mutex> lock(m);

        cv.wait(lock, [] { return counter % 2 == 0; });
        -----
        cv.notify_all();
    }
}
```

✓ After printing → increment → wake up odd thread.

✓ 2 Producer-Consumer Using Condition Variables

Classic bounded buffer (queue) problem.

```
#include <iostream>
#include <queue>
#include <thread>
#include <mutex>
#include <condition_variable>

std::queue<int> q;
std::mutex m;
std::condition_variable cv;
const int MAX_SIZE = 5;

void producer() {
    int value = 0;
    while (true) {
        std::unique_lock<std::mutex> lock(m);
        cv.wait(lock, [] { return q.size() < MAX_SIZE; });

        std::cout << "Produced: " << value << std::endl;
        q.push(value++);

        cv.notify_all();
    }
}

void consumer() {
    while (true) {
        std::unique_lock<std::mutex> lock(m);
        cv.wait(lock, [] { return !q.empty(); });

        int val = q.front();
        q.pop();
        std::cout << "Consumed: " << val << std::endl;

        cv.notify_all();
    }
}

int main() {
    std::thread p(producer);
    std::thread c(consumer);

    p.join();
    c.join();
}
```